

C++ Foundations

From tests to memory to objects

TEST

CODE

MEMORY

CLASS

4-HOUR SESSION

Today's path

One continuous story: prove it, write it, understand its memory, then package it into an object.

1 Test first

findMax + debugger
TDD mindset

2 Baby steps

two money problems
loops + functions

3 Memory

stack, heap, pointers
leaks + Valgrind

4 Classes

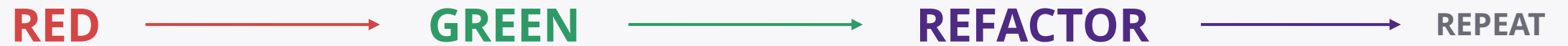
dynamic array
constructor + destructor

Goal: leave able to reason about what C++ is doing, not just memorize syntax.

SECTION 1 • TESTING MINDSET

Start by proving your code works

A passing test only proves that one test case passed.



The important habit is not “tests exist.” It is “I am responsible for proving my code.”

WARM-UP

Yesterday: max of two values

We already wrote a tiny decision. Today we turn it into something testable.

```
int max2(int a, int b) {  
    if (a > b) {  
        return a;  
    }  
    return b;  
}
```

Question

How do we know this function is correct?

First answer

Run a few examples.

Better answer

Make the examples executable tests.

Step 1: write the test before the function

The first failure may simply be that the code does not compile yet. That still tells us the behavior is missing.

```
#include <cassert>

int main() {
    int values[] = {42};
    assert(findMax(values, 1) == 42);
}
```

Expected result

Build fails: findMax does not exist yet.

Why this is useful

The test states the behavior before the implementation.

RED

We have evidence that the behavior is not there.

TDD • GREEN

Step 2: write the minimum code that passes

Do not solve every future case yet. Make the current test green.

```
int findMax(const int values[], int size) {  
    return values[0];  
}
```

```
int values[] = {42};  
assert(findMax(values, 1) == 42);
```

Result

Test passes.

Important

Passing this test does not mean findMax is finished. It means this behavior works.

A passing test is evidence, not proof of all possible behavior.

HIDDEN DEFECT

Now make it look more complete

This version loops and compares, but contains a subtle defect.

```
int findMax(const int values[], int size) {  
    int max = values[0];  
  
    for (int i = 1; i < size; i++) {  
        if (values[i] > max) {  
            // BUG: forgot to update max  
        }  
    }  
  
    return max;  
}
```

```
int a[] = {42};  
assert(findMax(a, 1) == 42);  
  
int b[] = {9, 3, 1};  
assert(findMax(b, 3) == 9);
```

Both tests pass. Are we done?

TEST DESIGN

Add one case that challenges the assumption

Move the maximum away from the first position.

```
int c[] = {1, 9, 3};  
assert(findMax(c, 3) == 9);
```

Result

Assertion fails.

Lesson

The old tests were not wrong. They were incomplete.

Before fixing the bug, add more tests so we understand the behavior we want.

OWNERSHIP

Before fixing, strengthen the test set

Think about positions, signs, duplicates, and small inputs.

```
int one[] = {7};  
assert(findMax(one, 1) == 7);  
  
int first[] = {9, 3, 1};  
assert(findMax(first, 3) == 9);  
  
int middle[] = {1, 9, 3};  
assert(findMax(middle, 3) == 9);  
  
int last[] = {1, 3, 9};  
assert(findMax(last, 3) == 9);  
  
int neg[] = {-8, -2, -10};  
assert(findMax(neg, 3) == -2);
```

Position

first / middle / last

Values

positive / negative / duplicate

Size

smallest valid input

DEBUGGING

Do not guess: stop inside the bug

Put a breakpoint inside the if and inspect values[i] and max.

```
if (values[i] > max) {  
    // breakpoint here  
    // values[i] == 9  
    // max      == 1  
}
```

Observe

The condition is true.

Then step

max does not change.

Diagnosis

The comparison is correct. The state update is missing.

TDD • GREEN

Fix the defect, then rerun everything

The change is one line. The confidence comes from the whole test set.

```
if (values[i] > max) {  
    max = values[i];  
}
```

one element

PASS

max first / middle / last

PASS

negative values

PASS

The professor's tests are not your quality strategy. Your own reasoning is.

SECTION 2 • BABY STEPS

Now just write some C++

Two small money problems to practice variables, loops, floating-point values, and functions.

Problem A

\$1,000,000 today
vs.
\$0.01 doubled every day

Problem B

How long until your savings reach \$1,000,000?

These are intentionally simple. The goal is to get everyone typing and thinking.

PROBLEM A

Which would you choose?

Take \$1,000,000 now, or take one cent and double the amount every day for 30 days?

\$1,000,000

OR

\$0.01 × 2 × 2 × ...

Before coding

Vote. Then guess the final amount.

PROBLEM A

One file. One loop.

Keep this one intentionally small

```
#include <iostream>
using namespace std;

int main() {
    double money = 0.01;

    for (int day = 1; day < 30; day++) {
        money *= 2;
    }

    cout << money << endl;
}
```

Watch the loop

Why is the condition `day < 30`?

Try changing it

What happens with `day <= 30`?

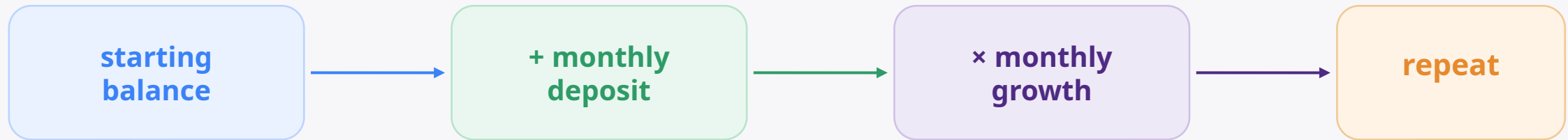
Vocabulary

off-by-one error

PROBLEM B

How long to reach \$1,000,000?

Start with a balance, add a monthly deposit, then apply monthly interest.



```
while (balance < target) {  
    balance += monthlyDeposit;  
    balance *= 1 + annualRate / 12.0;  
    months++;  
}
```

FUNCTIONS

Move the calculation into a function

ma

```
double yearsToTarget(double startingBalance,
                    double monthlyDeposit,
                    double annualRate,
                    double target) {
    double balance = startingBalance;
    int months = 0;

    while (balance < target) {
        balance += monthlyDeposit;
        balance *= 1 + annualRate / 12.0;
        months++;
    }

    return months / 12.0;
}
```

```
int main() {
    cout << yearsToTarget(1000, 500, 0.08,
1000000)
        << " years" << endl;
}
```

End of the warm-up: variables, loops, doubles, parameters, return values, functions.

SECTION 3 • MEMORY

C++ makes memory visible

Stack, heap, pointers, references, arrays, ownership, leaks.

STACK

Automatic lifetime
Function calls
Local variables

HEAP

Dynamic lifetime
new / delete
You manage ownership

POINTER

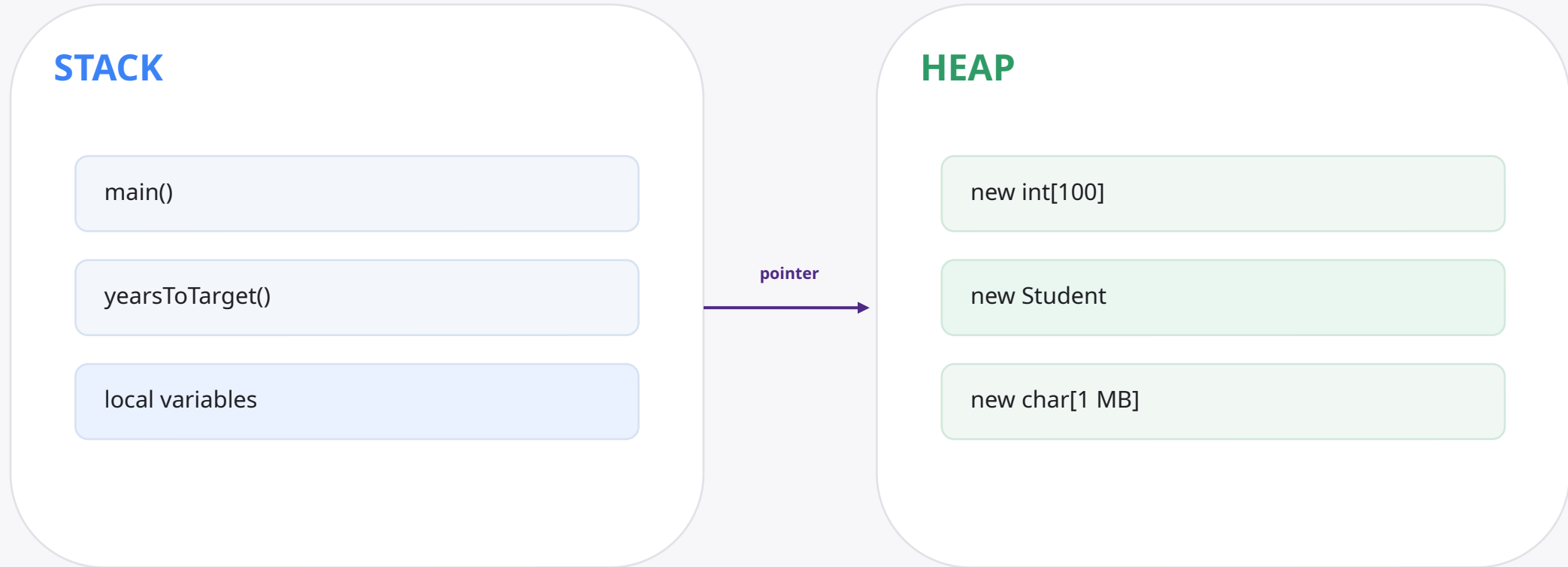
An address
Can be null
Can move through memory

This is the section where C++ starts feeling fundamentally different from Java.

MEMORY MODEL

Two regions, two lifetime models

Conceptual model only: exact process layout varies.



LIVE DEMO

How much stack do we have?

All

```
#include <iostream>
using namespace std;

void consumeStack(int depth) {
    volatile char block[1024 * 1024];
    block[0] = static_cast<char>(depth);

    cout << "depth: " << depth << endl;
    consumeStack(depth + 1);

    block[1] = static_cast<char>(depth); // discourage tail-call
    optimization
}

int main() {
    consumeStack(1);
}
```

Prediction

How many calls before it crashes?

Remember

The exact number varies by system and limits.

Linux

ulimit -s shows the stack limit in KB.

POINTERS + REFERENCES

Two ways to refer to an existing object

Both avoid copying. They have different syntax and guarantees.

```
int x = 10;
```

```
int* p = &x;  
*p = 20;
```

```
int& r = x;  
r = 30;
```

Pointer

- Stores an address
- Can be nullptr
- Can point somewhere else
- Use * to dereference

Reference

- Alias for an object
- Must refer to something
- Cannot be resealed
- Use like the object itself

For function parameters, compare pass-by-value, pass-by-pointer, and pass-by-reference.

FUNCTION PARAMETERS

Value, pointer, reference

Ask: do we need a copy, do we allow null, and do we intend to modify?

```
void byValue(int x) {  
    x++;  
}  
  
void byPointer(int* x) {  
    (*x)++;  
}  
  
void byReference(int& x) {  
    x++;  
}
```

byValue(x)

changes the local copy

byPointer(&x)

changes x through its address

byReference(x)

changes x through an alias

ARRAYS

Array syntax and pointer arithmetic meet

An array name is not literally a pointer, but it decays to a pointer in most expressions.

```
int a[] = {10, 20, 30};

cout << a[1] << endl;      // 20
cout << *(a + 1) << endl;  // 20

int* p = a;
cout << *p << endl;        // 10
p++;
cout << *p << endl;        // 20
```

Identity

`a[i]` is equivalent to `*(a + i)`

Important exception

`sizeof(a)` sees the actual array when `a` is still an array.

Function parameter

`int a[]` in a parameter behaves like `int* a`.

UNDEFINED BEHAVIOR

C++ does not protect raw array bounds

Java throws. Raw C++ arrays simply let you access memory you do not own.

```
int a[3] = {10, 20, 30};  
  
cout << a[0] << endl;  
cout << a[1] << endl;  
cout << a[2] << endl;  
cout << a[3] << endl;    // undefined behavior
```

Possible result

prints garbage

Possible result

appears to “work”

Possible result

crashes

The dangerous case is often the one that appears to work.

HEAP ALLOCATION

new gives you memory. delete gives it back.

With raw heap memory, ownership is your responsibility.

```
int* p = new int;  
*p = 42;  
cout << *p << endl;  
delete p;  
  
int* a = new int[100];  
a[0] = 7;  
delete[] a;
```

Single object

new T → delete p

Array

new T[n] → delete[] p

Mismatch

Wrong delete form is a bug.

MEMORY LEAK

When the process exits, the OS reclaims its memory

The real danger is a process that keeps running while unreachable allocations accumulate.

```
void leak() {  
    int* data = new int[1000];  
    data[0] = 42;  
    // forgot: delete[] data;  
}
```

Short program

The OS reclaims process memory at exit.

Long-running service

Leak repeats for hours / days / months.

Eventually

Memory pressure, slowdown, OOM, crash.

TOOLING

Valgrind can find leaks that tests miss

Your output can be correct and your memory behavior can still be wrong.

```
g++ -g leak.cpp -o leak
valgrind --leak-check=full ./leak
```

Look for

definitely lost

Also useful

invalid read / invalid write

Mindset

Tests + memory tools answer different questions.

LIVE DEMO • DISPOSABLE VM ONLY

Watch a leak consume the machine

Use tmux so the program, memory monitor, and kernel log are visible at the same time.

```
tmux

# split vertically
Ctrl-b %

# split horizontally
Ctrl-b "
```

Pane 1

run the leaking program

Pane 2

watch -n 0.5 free -h

Pane 3

sudo dmesg -w (or sudo journalctl -kf)

Do this only on a disposable teaching VM. Not on a shared or production machine.

LIVE DEMO

Allocate, touch, keep every block

To

```
#include <cstring>
#include <iostream>
#include <vector>
using namespace std;

int main() {
    const size_t MB = 1024 * 1024;
    const size_t blockSize = 100 * MB;
    vector<char*> blocks;
    size_t total = 0;

    while (true) {
        char* p = new char[blockSize];
        memset(p, 1, blockSize);
        blocks.push_back(p);
        total += 100;
        cout << total << " MB allocated" << endl;
    }
}
```

Observe

available memory falls

Eventually

the process may be killed by the OOM killer

Afterward

sudo dmesg -T | tail -50

EDGE CASE

Sometimes lifetime is shorter than cleanup

A provocative systems example: if a short-lived process is guaranteed to terminate before memory pressure matters, cleanup may not be on the critical path.

Normal software

Allocate
Use
Release
Repeat

Time-critical loop

Allocation / deallocation can add cost and timing variability.

Better pattern

Pre-allocate a pool before the critical loop, then reuse it.

The lesson is not “leaks are fine.” The lesson is that lifetime and timing requirements drive memory strategy.

MEMORY STRATEGY

Allocate repeatedly vs. allocate once and reuse

A tiny benchmark can make the idea visible, but exact timing varies by machine and build settings.

```
for (int i = 0; i < N; i++) {  
    char* p = new char[1024];  
    p[0] = static_cast<char>(i);  
    delete[] p;  
}
```

Repeated

new / delete each iteration

```
char* pool = new char[1024];  
for (int i = 0; i < N; i++) {  
    pool[0] = static_cast<char>(i);  
}  
delete[] pool;
```

Reuse

one allocation outside the loop

SECTION 4 • CLASSES

Java made some memory management invisible

Let's demystify one familiar object: ArrayList.

```
ArrayList<Integer> values = new ArrayList<>();
```

```
values.add(10);    values.add(20);    values.add(30);    ...
```

Question

If an array has a fixed size, why does ArrayList feel unlimited?

JAVA MENTAL MODEL

What ArrayList is doing conceptually

Simple ArrayList Implementation (Simplified)

```
class SimpleArrayList<E> {  
    Object[] data = new Object[2];  
    int size = 0;  
  
    void add(E value) {  
        if (size == data.length) {  
            Object[] bigger = new Object[data.length * 2];  
            System.arraycopy(data, 0, bigger, 0, data.length);  
            data = bigger;  
        }  
        data[size++] = value;  
    }  
}
```

Capacity

physical array size

Size

number of elements stored

Growth

allocate bigger + copy + replace

C++

Build our own tiny dynamic array

This is the same core idea, except raw memory ownership is visible.

```
class DynamicArray {  
private:  
    int* data;  
    int size;  
    int capacity;  
  
public:  
    DynamicArray();  
    ~DynamicArray();  
  
    void add(int value);  
    int get(int index) const;  
};
```

data

points to heap memory

size

how many elements are in use

capacity

how many elements fit before growth

OBJECT LIFETIME

Constructor acquires. Destructor releases.

Tie the class directly back to the heap discussion.

```
DynamicArray::DynamicArray() {  
    size = 0;  
    capacity = 2;  
    data = new int[capacity];  
}  
  
DynamicArray::~~DynamicArray() {  
    delete[] data;  
}
```

Constructor

object becomes ready to use

Destructor

release owned heap memory

Forget delete[]?

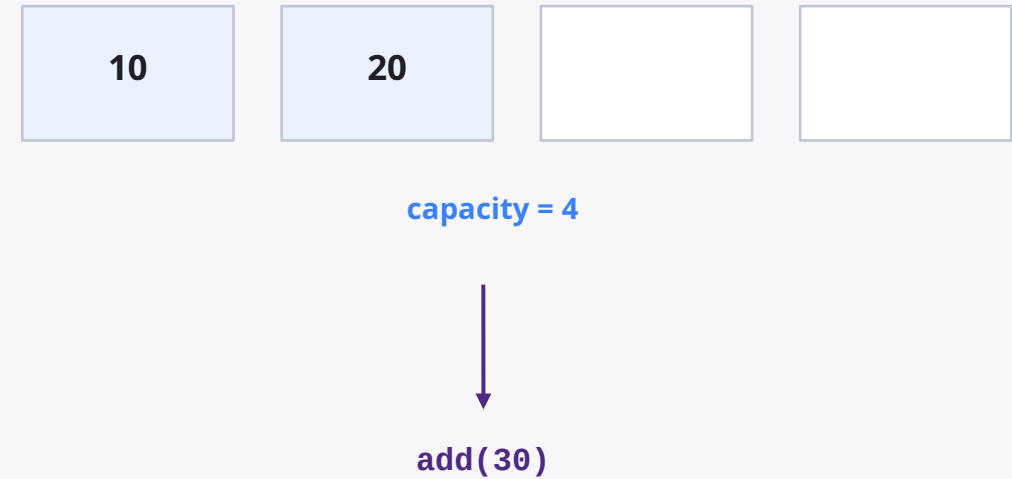
Valgrind reports the leak.

CAPACITY GROWTH

When full: allocate bigger, copy, release old

Th

```
void DynamicArray::add(int value) {  
    if (size == capacity) {  
        int newCapacity = capacity * 2;  
        int* bigger = new int[newCapacity];  
  
        for (int i = 0; i < size; i++) {  
            bigger[i] = data[i];  
        }  
  
        delete[] data;  
        data = bigger;  
        capacity = newCapacity;  
    }  
    data[size++] = value;  
}
```



CLOSE THE LOOP

A class can pass its functional tests and still leak

Run the same memory tool against the object.

```
DynamicArray::~DynamicArray() {  
    // delete[] data;    // accidentally  
    omitted  
}
```

```
g++ -g dynamic_array.cpp main.cpp -o demo  
valgrind --leak-check=full ./demo
```

Functional tests

add() and get() may all pass.

Memory test

Valgrind still finds the ownership bug.

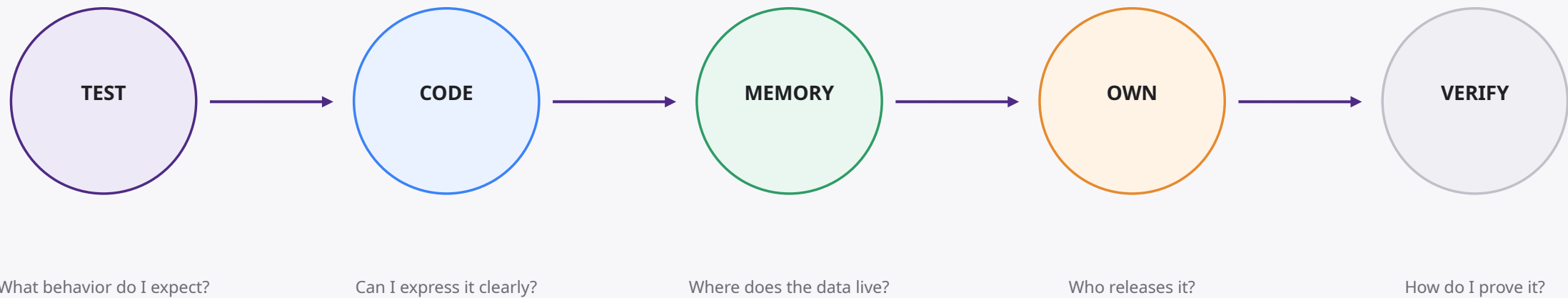
Mindset

Correctness includes resource behavior.

WRAP-UP

The complete C++ story

Everything today connects back to ownership and evidence.



C++ gives you control. The tradeoff is that you must reason about more of the machine.

From a passing test to a program you understand.

Tests • loops • functions • pointers • stack • heap • leaks • classes

Questions?

Today's checklist

- ✓ test first
- ✓ debug with evidence
- ✓ know stack vs heap
- ✓ use new/delete carefully
- ✓ check leaks
- ✓ connect memory to classes